

Ponovitev

classpath

module-path

vsi konstruktorji private?

dedovanje (extends) – med razredi

super()

@Override

super.imemetode()

abstract

interface

implements

1

super

V konstruktorjih podrazredov

- konstruktor brez argumentov je privzeti konstruktor, ki ga prožijo podrazredi, zato je klic **super()** odveč, saj za to avtomatično poskrbi prevajalnik, torej je klic **super (parametri)** aktualen le, če ga kličemo z argumenti
- če v podrazredu ne specificiramo konstruktorja, prevajalnik avtomatično kreira konstruktor brez argumentov, ki preprosto kliče **super()** (torej privzeti konstruktor nadrazreda)

Lahko za uporabo originalne metode podrazreda, ki smo jo sicer predefinirali ("override-ali")

```
@Override
public void dvig(int znesek) {
    if ((vrniStanje()-znesek)> dovoljeniMinus)
        super.dvig(znesek);
    else
        System.out.println(" Prevec hoces ");
}
```

2

Generalizacija

- včasih imajo razredi podobne odgovornosti
- identične metode
- **skupne** stvari damo v (splošnejši) **nadrazred**
- **specifične** pa ostanejo v **podrazredu**
- koncept: generalizacija/specializacija
- **dedovanje** – implementacijski konstrukt

3

Dedovanje

“inheritance”

- podrazredi nekega razreda **podedujejo metode** (in attribute nadrazreda)
- podrazred lahko predefinira podedovane metode
- definira pa lahko tudi nove, sebi lastne metode in attribute
- dedovanje (inheritance) zagotavlja mehanizem za organiziranje razredne hierarhije
- odpravlja nepotrebno podvajanje
- **nadrazred** (base class, superclass)
- **podrazred** (derived class, subclass)
- ločimo **enkratno** in **večkratno** dedovanje (med razredi Java podpira le enkratno dedovanje)

4

Dedovanje

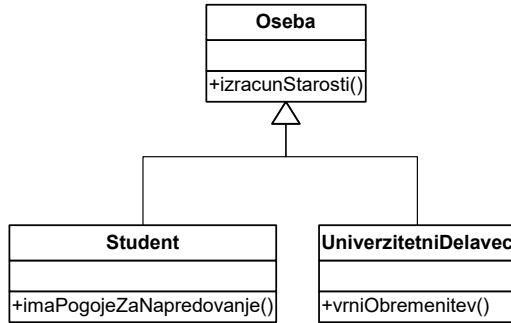
```

class Oseba {
...
    izracunStarosti() {...}
}

class Student extends Oseba {
...
    imaPogojeZaNapredovanje() {...}
}

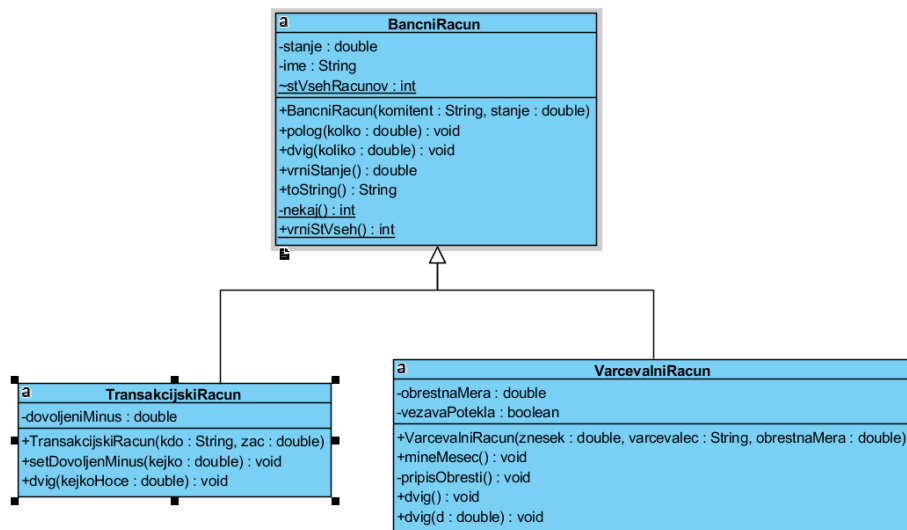
class UniverzitetniDelavec extends Oseba {
...
    vrniObremenitev() {...}
}

```



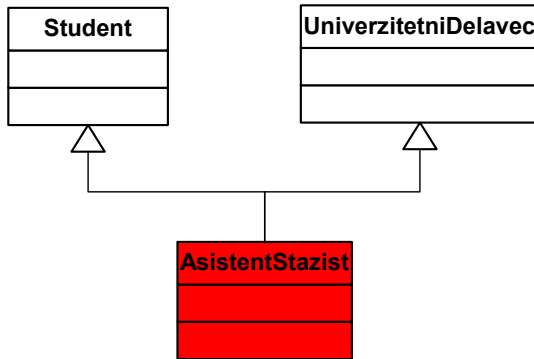
vsi razredi so izpeljani iz java.lang.Object

5

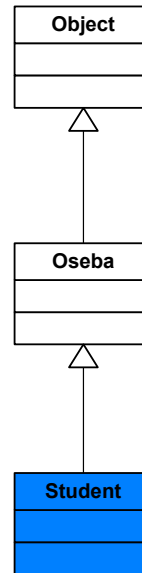


6

Razlikujmo!



večkratno dedovanje

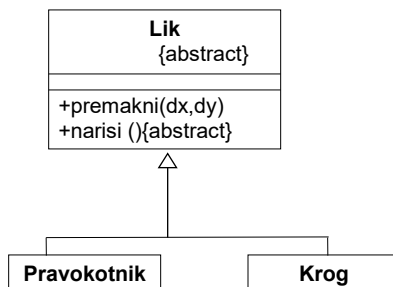


globina dedovanja

7

Abstraktni razred

Če ima kakšno abstraktno metodo (metodo, ki nima implementacije), je razred obvezno **abstrakten**!



Abstraktni razred
neposredno ne
more imeti
primerkov

Ni pa nujno, da ima kakšno abstraktno metodo,
pa ga označimo kot abstract.

8

Abstraktni razred Lik

```

abstract class Lik
{
    abstract double ploscina ();
}

class Trikotnik extends Lik {
    double b, h;
    Trikotnik (double b, double h)
    { this.b=b; this.h=h; }

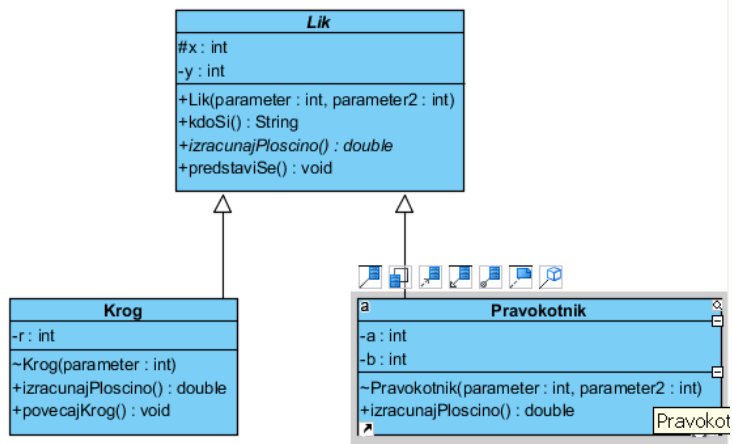
    double ploscina () {return 0.5*b*h;}
}

class Kvadrat extends Lik {
    double a;
    Kvadrat (double a) { this.a=a; }

    double ploscina () {return 0.5*a*a;}
}

```

9

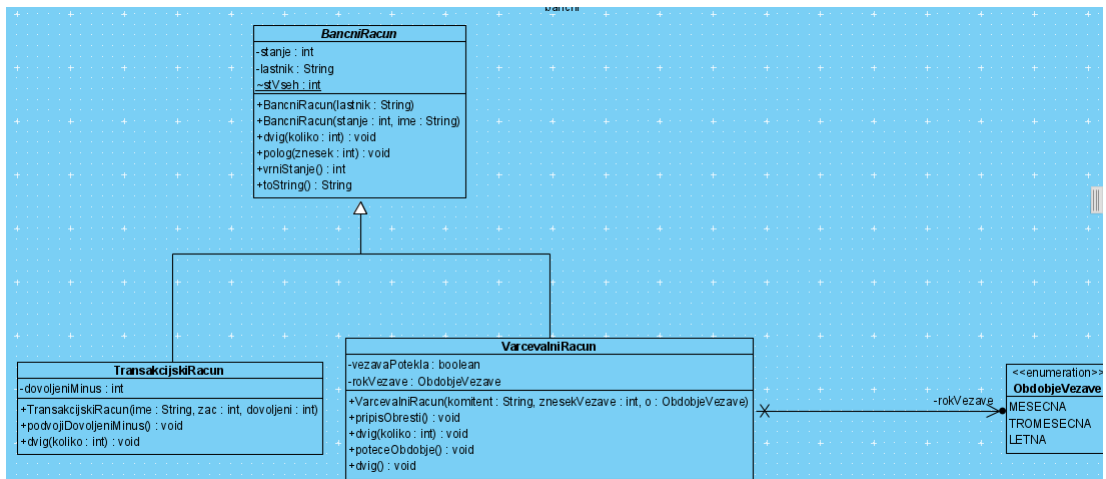


10

Abstraktni razredi, metode

- ne moremo ustvariti primerkov abstraktnega razreda ("instatiation" ni možna)
- abstraktna metoda nima telesa, signaturo zaključuje podpičje
- razred z abstraktno metodo **moramo** deklarirati kot abstrakten (abstract)
- podrazred abstraktnega razreda lahko instanciramo le, če ta implementira vse abstraktne metode, sicer je tudi podrazred abstrakten

11

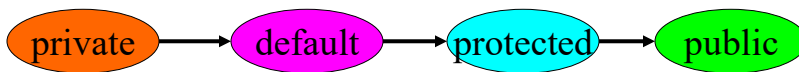


12

- neka spremenljivka (referenca) lahko predstavlja objekte različnih razredov,
- povsod, kjer imamo referenco splošnejšega tipa (narazreda) , lahko uporabimo tudi objekte (primerke) poljubnega podrazreda, saj ti zagotavljajo vse funkcionalnosti, ki so predpisane že v nadrazredu
- preko reference nadrazreda ne moremo prožiti oz. dostopati do metod, specifičnih za podrazrede

13

Predefiniranje dostopa za metode



vidnih – dostopnih pravic v podrazredih ne moremo zmanjševati !

14

Pomembni modifikatorji pri metodah

- **abstract**
ni implementacije, le signatura
- **final**
onemogoča spremembe
pri razredih onemogoči podrazrede
pri metodah - onemogoči predefiniranje
pri atributih – spreminjanje
pri parametrih – ni jim dodeliti nove vrednosti
- **static**
razredna metoda – znotraj razreda lahko uporablja le razredne attribute in metode
- Dodatno še: **synchronized**, **native**

15

Vse možne kombinacije modifikatorjev in elementov

modifikator	R	A	M	K	blok stavkov
public	D	D	D	D	N
protected	N	D	D	D	N
(nič)	D	D	D	D	D
private	N	D	D	D	N
final	D	D	D	N	N
abstract	D	N	D	N	N
static	D	D	D	N	D
native	N	N	D	N	N
transient	N	D	N	N	N
volatile	N	D	N	N	N
synchronized	N	N	D	N	D

R - razred
A - atribut
M - metoda
K - konstruktor

16

Vmesnik

“interface”

- običajno vsebuje le abstraktne metode
- ne moremo ga direktno uporabiti v smislu instanciranja
- definira (ne implementira) metode, ki jih kasneje razredi implementirajo

```
public interface IPromet {
    boolean zmanjsaj (float minus);
    boolean povecaj (float koliko);
    float stanje();
}
```

17

```
public interface INalozba {
    public abstract int vrniTrenutnoVrednost();
    void vrniIveganje();
    double vrniDonosnost();
}
```

```
public class Nepremicnina implements INalozba, Serializable {
```

```
public class VarcevalniRacun extends BancniRacun implements INalozba{
```

```
public class Kripto implements INalozba {
```

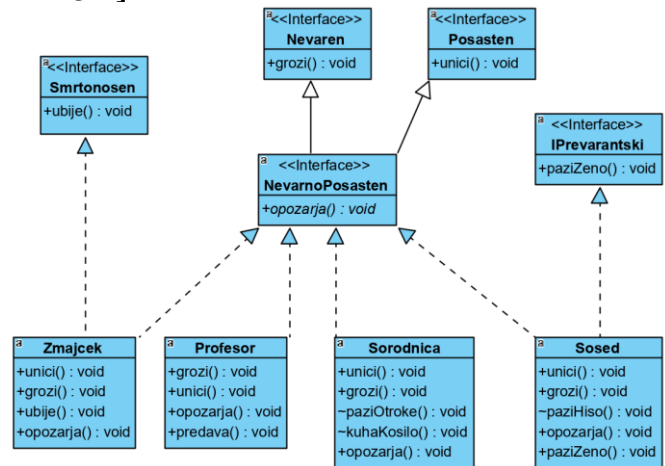
razredi lahko implementirajo **več kot le en vmesnik**

18

Hierarhija vmesnikov

[public] interface ImeVmesnika
[extends SeznamVmesnikov]

primer: [Grozljivka](#)



19

Pridobitev

- več sicer nesorodnih, nepovezanih razredov lahko implementira isti vmesnik
- na mestu, kjer so zahtevane neke storitve, lahko uporabim primerek poljubnega razreda oz. poljubni objekt, če le ta ima ustrezne metode oz. implementira ustrezni vmesnik

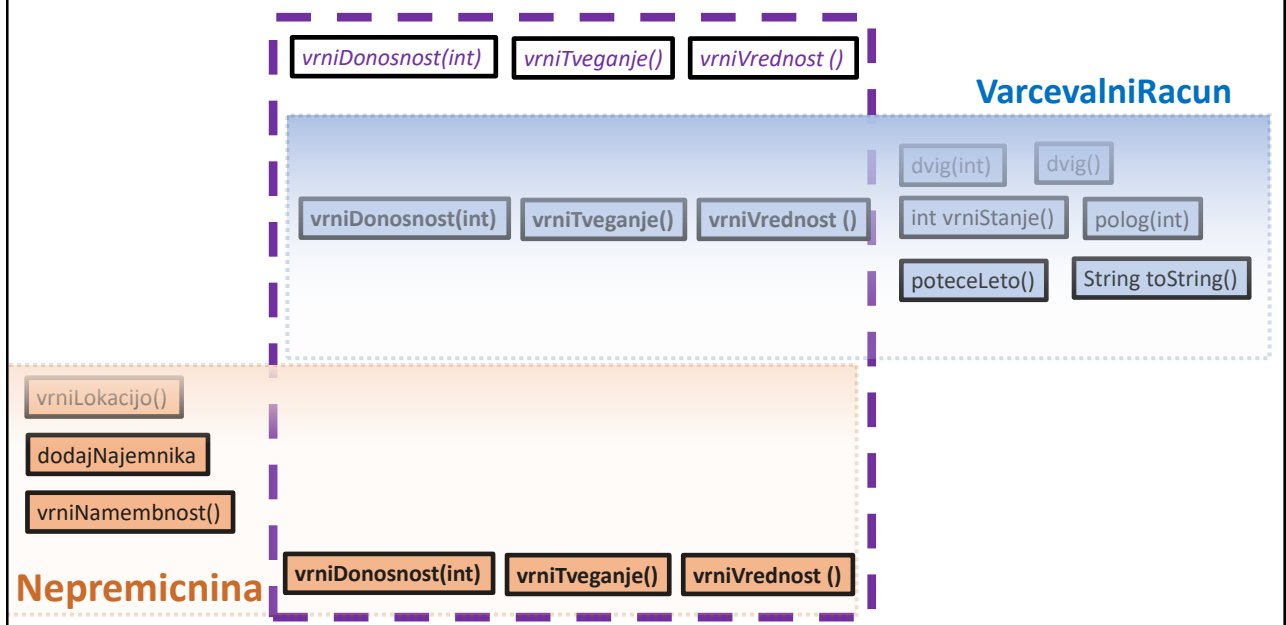
"on the fly integration"

20

- **neka spremenljivka (referenca) lahko predstavlja objekte različnih razredov**, saj lahko uporabimo oz. ji priredimo referenco na poljubni objekt (primerke) kateregakoli razreda, ki implementira ta vmesnik
- povsod, kjer imamo referenco splošnejšega tipa (vmesnika), lahko uporabimo tudi objekte (primerke) poljubnega razreda, ki ta vmesnik implementirajo, saj ti objekti zagotavljajo vse funkcionalnosti, ki so predpisane v vmesniku
- **preko reference vmesnika ne moremo prožiti oz. dostopati do dodatnih razreda specifičnih metod**

21

interface INalozba



22

Enotna obravnava

```
int skupajSredstev = 0;  
for (INalozba i : nalozbe)  
    skupajSredstev += i.vrniTrenutnoVrednost();
```

23

Vmesnik brez operacij?

- Ali ima smisel definirati vmesnik, v njem pa niti ene operacije?

Serializable, Clonable ...

24

```
System.out.println(vr instanceof INalozba);
```

```
if (enaInEdina instanceof Serializable)
    System.out.println("Lahko jo serializiram - tasco");
if (hiska instanceof Serializable)
    System.out.println("Lahko serializiram hisko");
else
    System.out.println(" Ne gre serializira hiske");
```

25

Primerjava

abstraktni razred	vmesnik
vsaj ena metoda abstraktna oz. razred označen kot abstract	vse metode abstraktne
podrazred extends	razredi implements
lahko več podrazredov	lahko ga implementira več razredov
enkratno dedovanje	večkratno dedovanje

26

Naprednejše teme glede vmesnika

morebitni atributi - **static final** (torej konstante)

morebitna privzeta implementacija – **default**

statične metode (običajno pomožne, „helper“) na nivoju vmesnika

prožimo: **ImeVmesnika**.imeStatičneMetode()